

Extend any Model Driven App form to surface a custom Copilot Studio agent through a Power Apps component framework control

Applies to: Dynamics 365 Sales, Dynamics 365 Customer Insights - Journeys, Dynamics 365 Customer Service, Power Apps, Copilot Studio

This reference architecture combines a Power Apps component framework (PCF) control, Copilot Studio, and Microsoft 365 Agents SDK to surface a custom agent in any Model Driven App (MDA) form.

Introduction

In a [previous article](#) we illustrated an integration and extensibility pattern between *Dynamics 365 Sales* and the *Copilot in Dynamics 365 Sales Agent* by creating a custom topic in the agent and using the Copilot APIs directly available in a PCF control to surface a custom user experience in the contact form.

While this pattern was interesting due to the relative simplicity of creating a custom topic in the existing first party agent and using the Copilot APIs in a PCF control to manage the integration, it also comes with some limits:

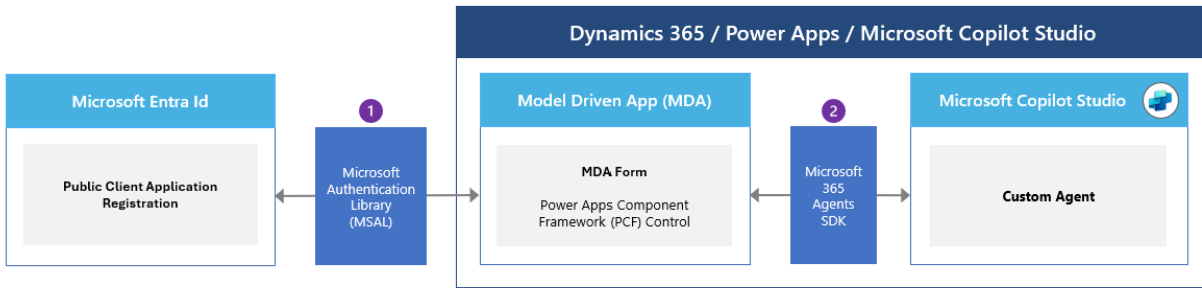
- It tightly couples with the existing first party agent (in this case *Copilot in Dynamics 365 Sales Agent*), which could link to behavior change or breakage as this agent evolves.
- The Copilot APIs directly available in a PCF control only work against the currently loaded Copilot for the current App (in that case *Dynamics 365 Sales*) and are still Preview.

Said otherwise, the same contact form surfacing the same PCF control would only work from *Dynamics 365 Sales* and not for instance from *Dynamics 365 Customer Insights - Journeys*, *Dynamics 365 Customer Service*.

The significant advantage of this new pattern is that it will allow to **extend any Model Driven App form to surface any custom Copilot Studio agent** with consistent behavior across all Apps and with proper architecture decoupling.

This is why this new pattern is strongly recommended from now on.

Architecture



Flow

The overall flow is relatively simple and will consist of two main steps

1. Obtain an authentication token to be able to use the Microsoft Copilot Studio APIs on behalf of the user
2. Establish a connection and communicate with your custom agent

Prerequisites

Public Client Application registration

This is required to allow the use of the Microsoft Copilot Studio APIs from the PCF control.

1. Go to Azure Portal and navigate to `Microsoft Entra ID → Manage → App registrations → New registration`
 - a. Give a self-explanatory name to your application (e.g. *"PCAforMCSAgent"*)
 - b. Choose `Accounts in this organizational directory only (xxxx - Single tenant)`
 - c. In the `Redirect URI` section choose `Single-page application (SPA)` for the platform, and your Dataverse organization main page for the redirect URI (e.g. *"https://orgXXXX.crm4.dynamics.com/main.aspx"*) - This can be added later if needed.

Microsoft Azure | Search resources, services, and docs (G+/)

Home > App registrations

Register an application

* Name

The user-facing display name for this application (this can be changed later).

PCAforMCSAgent

Supported account types

Who can use this application or access this API?

☒ Accounts in this organizational directory only (My organization only - Single tenant)

☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant)

☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)

☐ Personal Microsoft accounts only

[Help me choose...](#)

Redirect URI (optional)

We'll return the authentication response to this URI after successfully authenticating the user. Providing this now is optional and it can be changed later, but a value is required for most authentication scenarios.

Single-page application (SPA) | https://org[redacted].crm4.dynamics.com/main.aspx

Register an app you're working on here. Integrate gallery apps and other apps from outside your organization by adding from [Enterprise applications](#).

Microsoft Azure | Search resources, services, and docs (G+/)

Home > App registrations

PCAforMCSAgent

Deactivate Delete Endpoints Preview features

Got a second? We would love your feedback on Microsoft identity platform (previously Azure AD for developer). →

Overview

Quickstart

Integration assistant

Diagnose and solve problems

Manage

Support + Troubleshooting

Essentials

Display name : PCAforMCSAgent

Application (client) ID : 11d87077-[redacted]-915cb0950e1

Object ID : [redacted]

Directory (tenant) ID : 16d46994-[redacted]-ab3573835014

Supported account types : My organization only

Client credentials : [Add a certificate or secret](#)

Redirect URIs : 0 web, 1 spa, 1 public client

Application ID URI : [Add an Application ID URI](#)

Managed application in ... : PCAforMCSAgent

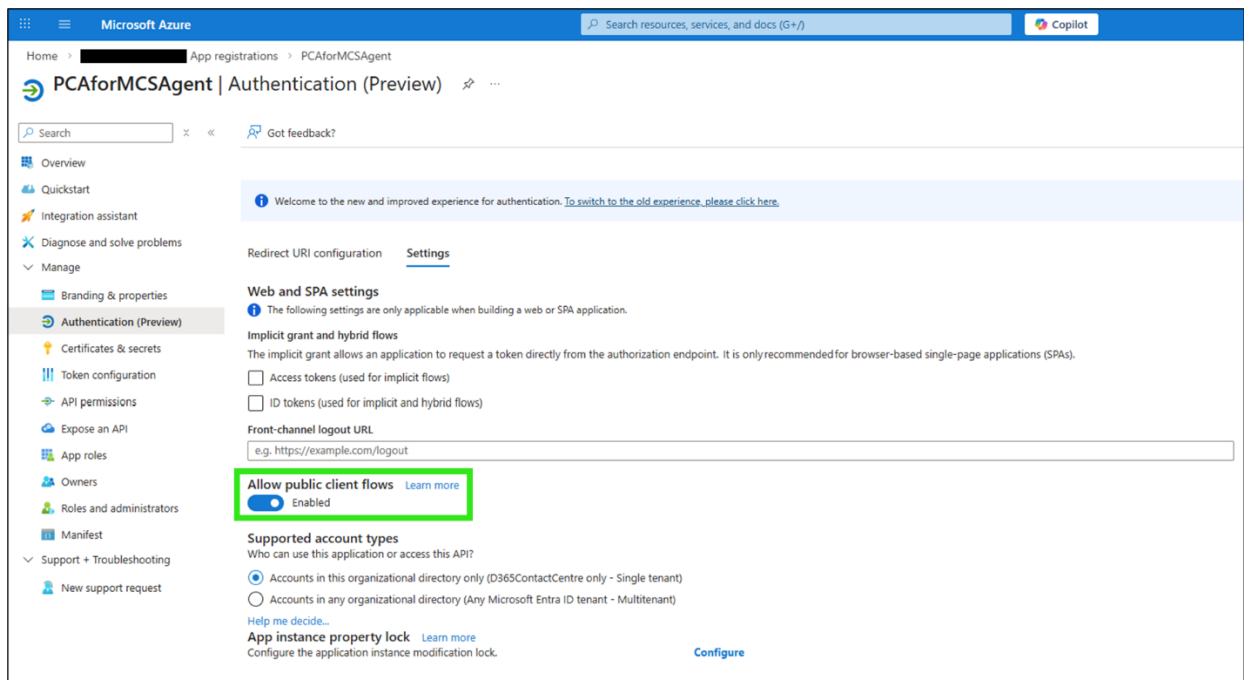
State : Activated



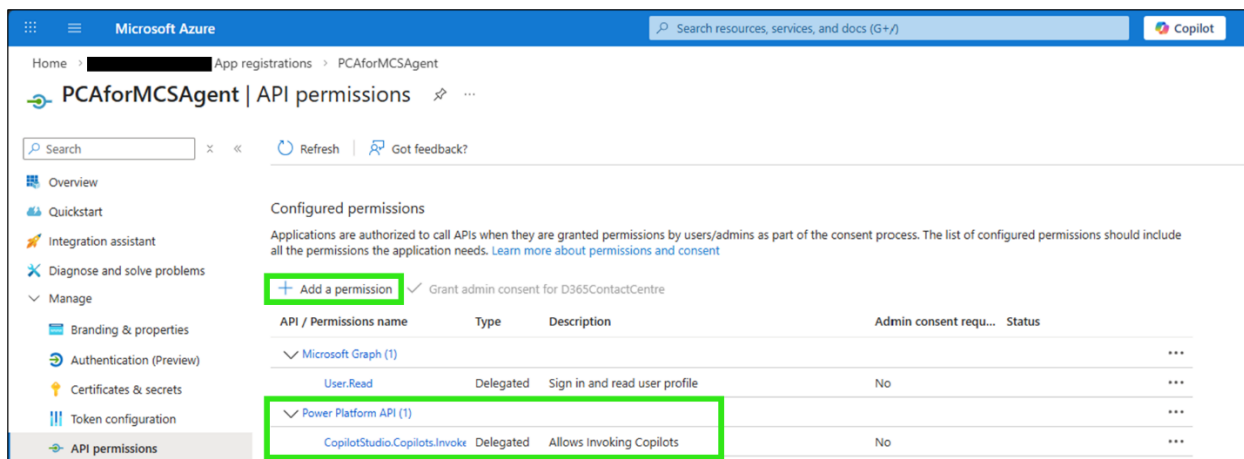
Take note of the following identifiers:

- Application (client) ID
- Directory (tenant) ID

- Once the Application registration is created, navigate to **Authentication** and from the **Settings** tab, switch the **Allow public client flows** toggle to enable.



- Then navigate to the API permissions section and Add a permission with the following: APIs my organization uses → Power Platform API → Delegated permissions → CopilotStudio → CopilotStudio.Copilots.Invoke



Microsoft Copilot Studio agent

This is the agent with which the PCF control will interact programmatically.

! Note

In the context of this article it doesn't matter what the exact purpose of the agent is.

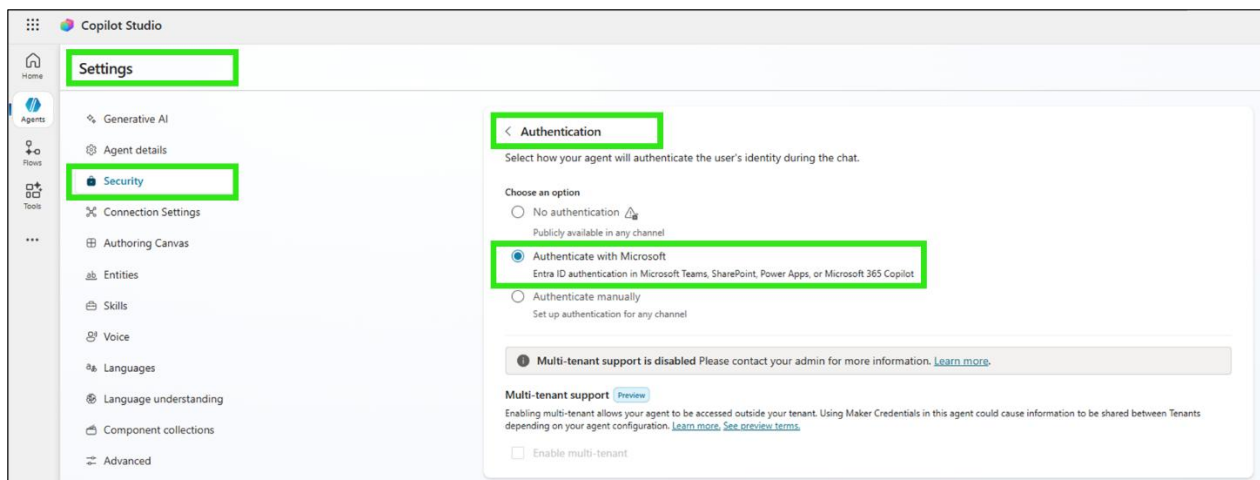
Though, as a general best practice, we would recommend avoiding chat-style scenarios in PCF controls and rather keeping those in the Copilot sidecar pane to maintain consistent user experience.

PCF controls are more adapted to custom user experience (such as the one described in the previous article mentioned in the introduction) and for this reason the agent should deliver repeatable results in a controlled format that can be interpreted and rendered by the PCF.

This can be achieved by providing appropriate instructions as part of the agent's definition so that the answer it provides is always adhering to a given predefined .json structure.

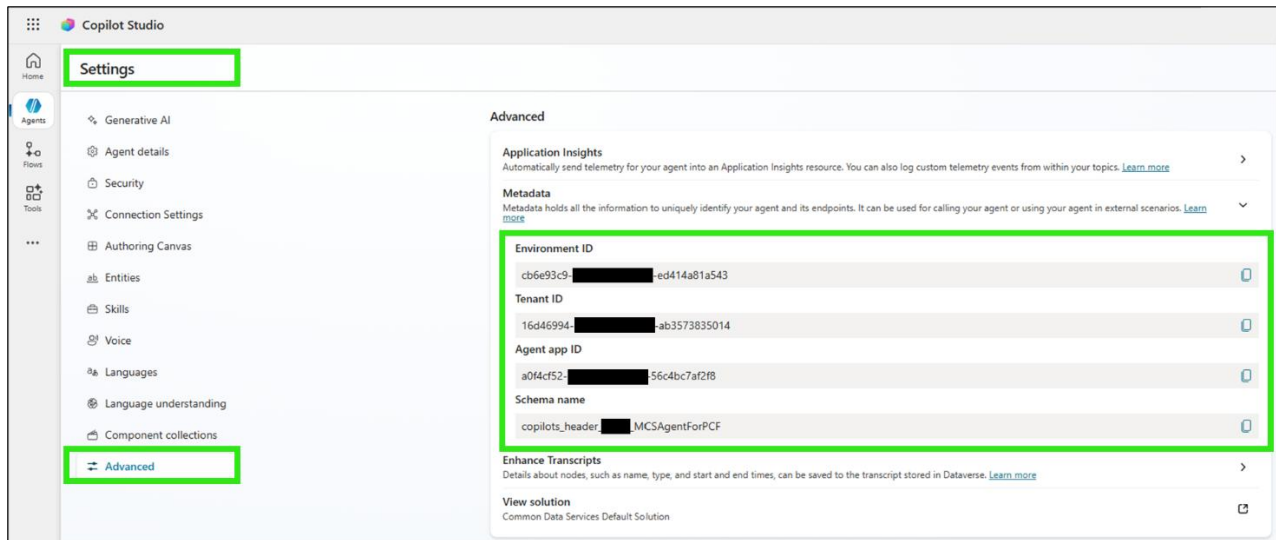
Authentication and metadata settings

In Copilot Studio, configure authentication by navigating to the agent settings, and in the Security section, choose “Authenticate with Microsoft” from the “Authentication” options.



Then, navigate to Advanced → Metadata and take note of the following identifiers:

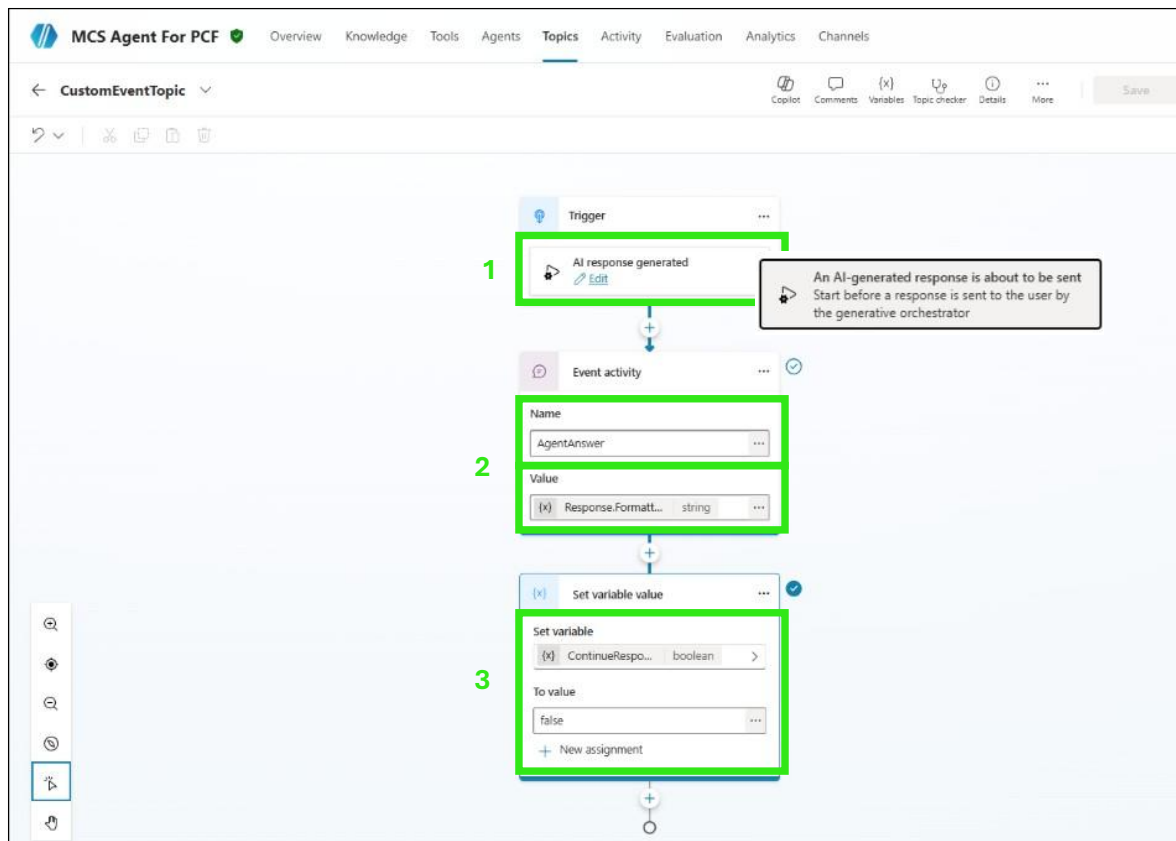
- Environment ID
- Tenant ID
- Agent app ID
- Schema name (note that the last part of the schema name can only be changed from default when first creating the agent with “advanced create”)



Topic configuration for programmatic integration

The goal of this integration pattern is to manage the back and forth with the agent programmatically through code logic and as such it's more appropriate to use an `event` activity rather than a `message` activity (like in a typical chat-based integration) as this will ease tagging, identifying, parsing and finally reacting to the activity with that code logic. More details on activity types: [Understanding the Activity Protocol | Microsoft Learn](#)

To achieve this, a simple approach is to create a Topic that will kick-in just before the agent returns its message and will send a properly tagged (i.e., named) event instead.



1. Set the Topic trigger to “AI response generated” so that the topic starts just before the response is about to be sent - [Set topic triggers - Microsoft Copilot Studio | Microsoft Learn](#)
2. Create an event activity with an easily identifiable name (e.g. here “AgentAnswer”) and set the value to the system variable `Response.FormattedText`.
3. Set the `ContinueResponse` variable to “false” to avoid redundant sending of the response activity as a message.

Implementation

Token acquisition

There are diverse ways to acquire a token to authenticate and call the Copilot APIs from your PCF control. The example below shows a straightforward approach using the [@azure/msal-browser](#) v4.xx package. If you build your PCF control with React, you can also use a wrapper library, depending on your app flow and requirements.

```
TypeScript Copy
/* MSALAuthentication.ts */
import { PublicClientApplication, Configuration } from "@azure/msal-browser";

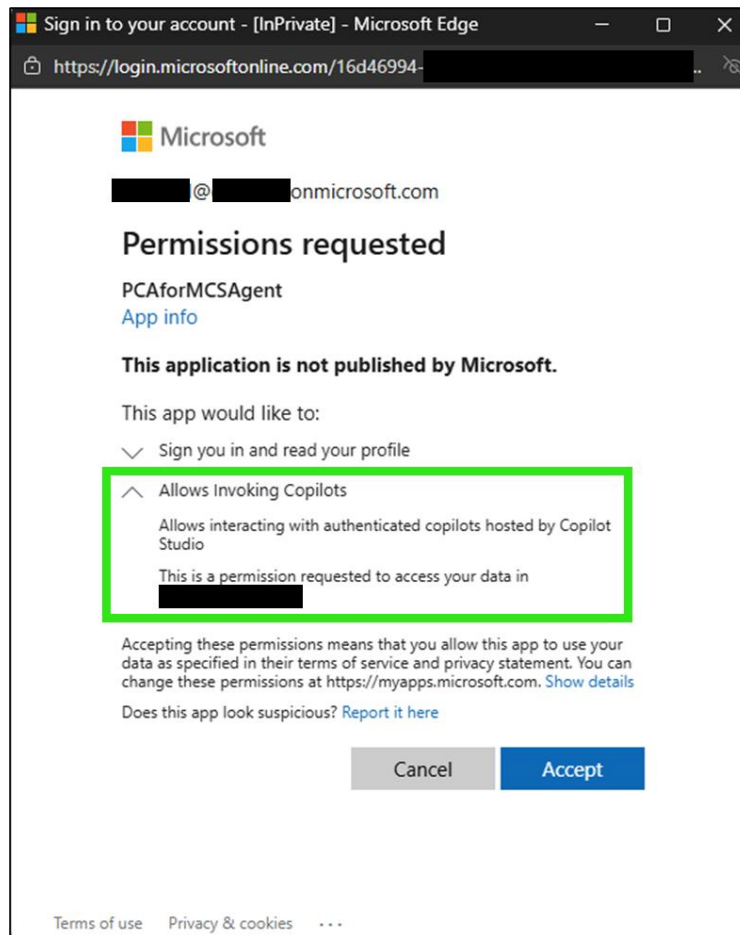
export function GetToken(): Promise<string> {

  const PCA_Configuration: Configuration = {
    auth: {clientId: "11d87077-xxxx-xxxx-xxxx-4915cb0950e1",
      authority: "https://login.microsoft.com/16d46994-xxxx-xxx-xxx-ab3573835014"},
    cache: {cacheLocation: "localStorage"}};

  const PCA = new PublicClientApplication(PCA_Configuration);

  return new Promise((resolve) => {
    PCA.initialize()
    .then(() => {
      const accessTokenRequest = {
        scopes: ["https://api.powerplatform.com/.default"],
        account: PCA.getAllAccounts()[0]??undefined};
      console.log("Token account ? : ");
      console.log(PCA.getAllAccounts()[0]);
      PCA.acquireTokenSilent(accessTokenRequest)
      .then((tokenSilentResponse) => {
        console.log("Got Token Silent");
        resolve(tokenSilentResponse.accessToken);
        return;
      })
      .catch((error) => {
        console.log(error);
        PCA.acquireTokenPopup(accessTokenRequest)
        .then((tokenPopupResponse) => {
          console.log("Got Token Popup");
          resolve(tokenPopupResponse.accessToken);
          return;
        })
        .catch((error) => {
          console.log(error);
        })
      })
    })
    return;
  })
  .catch((error) => {
    console.log(error);
  })
});
}
```

The first time a new user is using the control and triggers the acquisition of the token from the registered Public Client Application, a pop-up will ask for consent:



Next usages will either acquire the token silently with cache information, or briefly show the pop-up if cache was emptied in between, but without the need to reconfirm consent as this has been persisted in the registered Public Client Application.

Communicating with the agent

Authentication token along with connection settings are used to initiate and use a `CopilotStudioClient` from the [microsoft/agents-copilotstudio-client](#) package to communicate with your agent and get the expected formatted response.

TypeScript

Copy

```
/* MCSIntegration.ts */
import { GetToken } from './MSALAuthentication';
import { Activity } from '@microsoft/agents-activity';
import { ConnectionSettings, CopilotStudioClient } from '@microsoft/agents-copilotstudio-client';

export function GetMCSAnswer(ProgrammaticPrompt: string){

    const main = async () => {
        const MCSConnectionSettings: ConnectionSettings = {
            environmentId: "cb6e93c9-de7a-e613-8318-ed414a81a543",
            agentIdentifier: "copilots_header_cr994_MCSAgentForPCF"
        };
        const token = await GetToken();

        const copilotClient = new CopilotStudioClient(MCSConnectionSettings, token);

        let conversationId = ''
        for await (const act of copilotClient.startConversationStreaming(true)) {
            conversationId = act.conversation?.id ?? '';
            if (conversationId !== '')
            {
                const act = new Activity("message");
                act.text = ProgrammaticPrompt;
                act.conversation = { id: conversationId }
                for await (const reply of copilotClient.sendActivityStreaming(act)) {
                    processActivity(reply, conversationId, copilotClient);
                }
            }
        }

        function processActivity (act: Activity , convo: string, cc: CopilotStudioClient) {
            if (act.type == "event" && act.name == "AgentAnswer") {
                /*
                 Code logic to process the answer from the Agent in act.value
                */
            }
        }

        main().catch(e => console.log(e));
    }
}
```

Managing connector consent

If your agent uses a tool with a connector setup to use user's credentials (e.g., Dataverse connector) you'll need to manage the logic to surface the request to the user and provide the user's consent back to the agent.

Consent requests are sent as a message activity with an Adaptive Card as attachment which can be parsed and rendered by using the [AdaptiveCards](#) package.

In the example below this is done within a modal dialog (using a placeholder created in the main document), modify the previous `processActivity` function:

TypeScript

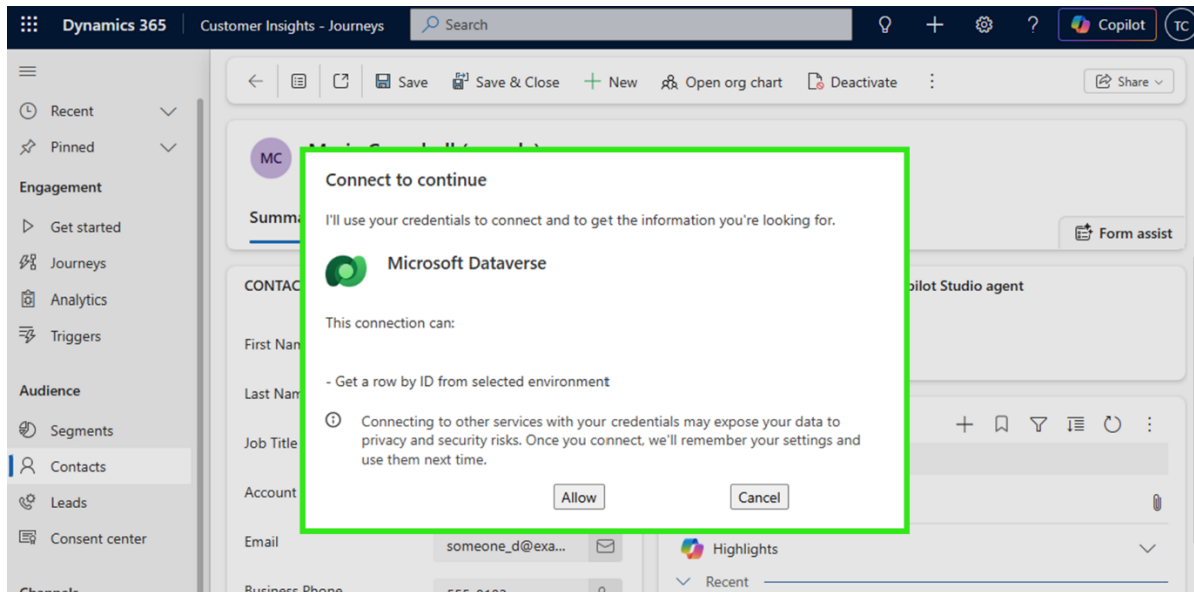
Copy

```
import * as AdaptiveCards from "adaptivecards";

[...]
```

```
function processActivity (act: Activity, convo: string, cc: CopilotStudioClient) {
  if (act.type == "event" && act.name == "AgentAnswer") {
    console.log("Received agent answer");
    /*
      Code logic to process the answer from the Agent
    */
  }
  else if (act.type == "message" && act.name == "connectors/consentCard") {
    const dialog = document.getElementById("placeholderdialog");
    const adaptiveCard = new AdaptiveCards.AdaptiveCard();
    const attachedAdaptiveCard = act.attachments;
    if (attachedAdaptiveCard != undefined && dialog != null
      && dialog instanceof HTMLDialogElement) {
      adaptiveCard.onExecuteAction = function(action) {
        processConsent(action, dialog, convo, cc).catch(e => console.log(e));
      }
      adaptiveCard.parse(attachedAdaptiveCard[0].content);
      const renderedCard = adaptiveCard.render();
      if (renderedCard != undefined) {
        dialog.innerHTML = "";
        dialog.appendChild(renderedCard);
        dialog.showModal();
      }
    }
  }
}
```

This could look like this once rendered in your App:



The last part is to define the `processConsent` function to handle the action events (i.e., Allow or Cancel) based on user input.

```
TypeScript Copy

async function processConsent ( action: AdaptiveCards.Action,
                                placeholderdialog: HTMLDialogElement,
                                convo: string,
                                cc: CopilotStudioClient) {

    if (action.title == "Allow") {
        const actConsent = new Activity("message");
        actConsent.conversation = { id: convo };
        actConsent.value = {action: "Allow", id: "submit", shouldAwaitUserInput: true};
        actConsent.channelData = {postBack: true, enableDiagnostics: true};
        console.log(actConsent);
        placeholderdialog.close();
        for await (const reply of cc.sendActivityStreaming(actConsent, convo)) {
            processActivity(reply, convo, cc);
        }
    }
    else if (action.title == "Cancel") {
        placeholderdialog.close();
    }
}
```